

// class notes

Backend Development

Node.js · Express · MongoDB · Authentication

fs

http

express

REST

mongoose

bcryptjs

JWT

```
const express = require("express");
const app = express();
app.use(express.json());
app.get("/products", (req, res) => {
  res.status(200).json({ data: products });
});
app.listen(3000, () => console.log("server..."));

const express = require("express");
const app = express();
app.use(express.json());
app.get("/products", (req, res) => {
  res.status(200).json({ data: products });
});
app.listen(3000, () => console.log("server..."));

const express = require("express");
const app = express();
app.use(express.json());
app.get("/products", (req, res) => {
  res.status(200).json({ data: products });
});
app.listen(3000, () => console.log("server..."));
```

9 units · syntax, examples, outputs, key points and common mistakes

Contents

What is covered in these notes, in teaching order.

- 01 Introduction to Node.js**
What Node.js is, how it differs from the browser, running files, modules and npm.
- 02 Node.js File System (fs Module)**
Creating, reading and updating files and folders with the built-in fs module.
- 03 Creating a Server with Node.js**
How a server works and how to build one using only the built-in http module.
- 04 Express Basics**
Building servers with Express: routes, responses, params, query and body.
- 05 REST API and CRUD**
A complete CRUD API on a products array, status codes and CORS.
- 06 MongoDB with Mongoose**
Storing users in MongoDB using Mongoose models.
- 07 Authentication and Authorization**
Signup, login, password hashing, JWT tokens and protected routes.
- 08 Forgot and Reset Password**
Complete forgot-password flow with a random token, expiry and email link.
- 09 Error Handling with try / catch**
Handling runtime errors safely with try / catch.

How to read these notes

● ● ●

JavaScript

```

1 // Code you write (with line numbers)
2 let app = express();

```

● ● ●

Terminal / Output

What the terminal / browser shows

★ KEY POINTS

- Points to remember

⚠ COMMON MISTAKES

- Errors students commonly make

01

UNIT 1

Introduction to Node.js

What Node.js is, how it differs from the browser, running files, modules and npm.

IN THIS UNIT

[1.1 What is Node.js](#)[1.2 Browser vs Node.js](#)[1.3 Running JavaScript with Node.js](#)[1.4 Modules and require\(\)](#)[1.5 npm and package.json](#)

1.1 What is Node.js

◆ WHAT IT IS

Node.js is a **JavaScript runtime environment**: it runs JavaScript **outside the browser**, on a computer or a server. It is built on **V8**, the same JavaScript engine used inside Google Chrome.

Node.js is

A runtime that executes JavaScript files

Used for backend: servers, APIs, database work, file handling

Node.js is NOT

A new programming language (the language is still JavaScript)

A framework (Express is a framework built on Node.js)

★ KEY POINTS

- Before Node.js (2009), JavaScript could run **only inside a browser**.
- Runtime = **JavaScript engine (V8)** + **extra features** (like **fs**, **http**, **crypto**) that the browser does not give.
- Node.js handles slow tasks (reading files, database calls, network requests) **asynchronously**, so one waiting task does not stop the others.

1.2 Browser vs Node.js

◆ WHAT IT IS

The JavaScript **language** is the same in both. The difference is the **environment**: what features are available around the language.

Point**Browser****Node.js**

Where code runs

Inside a web browser (Chrome, Firefox)

On a computer / server, from the terminal

Main use

Frontend: pages, buttons, UI

Backend: servers, APIs, databases, files

How code is run

Opening an HTML page with a `<script>` tag

`node app.js` in the terminal

Point	Browser	Node.js
Global object	<code>window</code>	<code>global</code>
DOM (<code>document</code>)	Available	Not available
<code>alert()</code> , <code>localStorage</code>	Available	Not available
File system access	Not allowed (security)	Allowed with the <code>fs</code> module
Create a server	Not possible	Possible with <code>http</code> / Express
Modules	<code><script></code> tags, <code>import</code>	<code>require()</code> (and <code>import</code>)
Environment decided by	The user (which browser / version)	The developer (chosen Node version)

Common to both: variables, functions, arrays, objects, `console.log()`, `setTimeout()`, Promises, `async` / `await`.

`</>` EXAMPLE: RUN THIS WITH NODE.JS

```

1 console.log(typeof window);
2 console.log(typeof global);
3 document.getElementById("btn");

```

► OUTPUT

```

undefined
object
ReferenceError: document is not defined

```

⚠ COMMON MISTAKES

- Using browser-only code (`document`, `window`, `alert`, `localStorage`) in a Node.js file. It gives a `ReferenceError`.

1.3 Running JavaScript with Node.js

{ } SYNTAX

```

1 node -v           // check Node.js is installed (prints version)
2 npm -v           // check npm is installed
3 node app.js       // run the file app.js

```

</> EXAMPLE: APP.JS

```

1 let name = "Aman";
2 console.log("Hello " + name);
3 console.log(10 + 20);

```

▶ OUTPUT

```

$ node app.js
Hello Aman
30

```

The output appears in the **terminal**, not in a browser.

1.4 Modules and require()

◆ WHAT IT IS

A **module** is a separate file or package of code. `require()` loads a module into the current file. There are three types.

Type	Examples	Install needed?	How to load
Built-in (core)	<code>fs</code> , <code>http</code> , <code>crypto</code> , <code>path</code>	No, comes with Node	<code>require("fs")</code>
Third-party (npm)	<code>express</code> , <code>mongoose</code> , <code>bcryptjs</code>	Yes, <code>npm install name</code>	<code>require("express")</code>
Own (local) file	<code>math.js</code> , <code>db/db.js</code>	No, you write it	<code>require("./math.js")</code>

{ } SYNTAX

```

1 // in the file that SHARES code
2 module.exports = { thing1, thing2 };
3
4 // in the file that USES code
5 let { thing1, thing2 } = require("./fileName.js");

```

</> EXAMPLE

```

1 // math.js
2 let add = (a, b) => a + b;
3 let sub = (a, b) => a - b;
4 module.exports = { add, sub };
5
6 // app.js
7 let { add, sub } = require("./math.js");
8 console.log(add(5, 3));
9 console.log(sub(5, 3));

```

▶ OUTPUT

```
8
2
```

★ KEY POINTS

- Own files need `./` (or `../`) at the start of the path. Without it, Node searches only built-in modules and `node_modules`.
- Only what is put in `module.exports` can be used by another file.

⚠ COMMON MISTAKES

- Forgetting `module.exports`: `require()` returns an empty object `{}`, and calling `add()` gives `TypeError: add is not a function`.
- Writing `require("math.js")` without `./` gives `Cannot find module 'math.js'`.

1.5 npm and package.json

◆ WHAT IT IS

npm (Node Package Manager) installs third-party packages. **package.json** is the project's information file: name, version and the list of installed packages (**dependencies**).

{ } SYNTAX

```
1 npm init -y           // create package.json with default values
2 npm install express   // install a package
3 npm install           // install everything listed in package.json
```

</> Example: package.json after installing express

```
1 {
2   "name": "backend-class",
3   "version": "1.0.0",
4   "main": "index.js",
5   "dependencies": {
6     "express": "^5.1.0"
7   }
8 }
```

File / Folder	What it is
<code>package.json</code>	Project details and the list of dependencies
<code>node_modules/</code>	Folder with the actual downloaded package code (very large)

File / Folder	What it is
<code>package-lock.json</code>	Exact versions of every installed package

★ KEY POINTS

- `node_modules` is **not** shared or uploaded. Anyone can recreate it by running `npm install`, because `package.json` lists every dependency.

≡ UNIT SUMMARY

<code>node -v</code>	Check Node.js version
<code>node app.js</code>	Run a JavaScript file
<code>npm init -y</code>	Create package.json
<code>npm install express</code>	Install a package
<code>require("fs")</code>	Load a built-in module
<code>require("./math.js")</code>	Load your own file (needs <code>./</code>)
<code>module.exports = { add, sub };</code>	Share code from a file

02

UNIT 2

Node.js File System (fs Module)

Creating, reading and updating files and folders with the built-in fs module.

IN THIS UNIT

[2.1 What is the fs Module](#)

[2.2 Sync Methods](#)

[2.3 Async Methods \(Callbacks\)](#)

2.1 What is the fs Module

◆ WHAT IT IS

`fs` (File System) is a **built-in** Node.js module used to create, read, update and delete files and folders. It comes with Node.js, so no installation is needed.

{ } SYNTAX

```
1  const fs = require("fs");
```

Every fs task comes in two versions: a **sync** version (name ends with `Sync`) and an **async** version (takes a callback).

2.2 Sync Methods

◆ WHAT IT IS

Sync methods **block** the program. The next line runs only after the current task is fully finished, so the code always runs in the written order.

{ } SYNTAX

Method	What it does
<code>fs.mkdirSync("folder")</code>	Creates a folder
<code>fs.writeFileSync("path", data)</code>	Creates a file with data. If the file exists, it is overwritten
<code>fs.readFileSync("path")</code>	Reads a file and returns its content as a Buffer
<code>fs.appendFileSync("path", data)</code>	Adds data to the end of a file

</> EXAMPLE

```
JavaScript
1  const fs = require("fs");
2
3  // 1. Create folder
4  fs.mkdirSync("students");
5  console.log("Folder created");
6
7  // 2. Create file with 4 names (\n = new line)
8  fs.writeFileSync("students/data.txt", "Rohit\nAman\nRahul\nRaju");
9  console.log("File created");
10
11 // 3. Read file -> returns Buffer, so convert with toString()
12 const data = fs.readFileSync("students/data.txt");
13 console.log("Current Data:");
14 console.log(data.toString());
15
16 // 4. Add one more name at the end
17 fs.appendFileSync("students/data.txt", "\nPriya");
18 console.log("Data added");
19
20 // 5. Read again
21 const updatedData = fs.readFileSync("students/data.txt");
22 console.log("Updated Data:");
23 console.log(updatedData.toString());
```

▶ OUTPUT

```
Folder created
File created
Current Data:
Rohit
Aman
Rahul
Raju
Data added
Updated Data:
Rohit
Aman
Rahul
Raju
Priya
```

★ KEY POINTS

- `readFileSync` returns a **Buffer** (raw bytes). Use `.toString()` or pass `"utf8"` as the second argument to get text: `fs.readFileSync("path", "utf8")`.
- `writeFileSync` replaces old content. `appendFileSync` keeps old content and adds new content.
- Output always comes in the same order as the code.

⚠ COMMON MISTAKES

- Running `mkdirSync("students")` again when the folder already exists gives an **EEXIST** error.
- Printing `data` without `.toString()` shows `<Buffer 52 6f 68 ...>` instead of text.

2.3 Async Methods (Callbacks)

◆ WHAT IT IS

Async methods **do not block**. Node starts the task and moves to the next line immediately. When the task finishes, Node runs the **callback** function that was passed.

{ } SYNTAX

```
1 fs.methodName(path, [data], (err, result) => {
2   if (err) {
3     console.log(err.message); // task failed
4     return;
5   }
6   // task succeeded, use result
7 });
```

This is called an **error-first callback**: the first parameter is always `err`. If the task succeeds, `err` is `null`.

</> EXAMPLE

```
1 fs.readFile("students/data.txt", (err, data) => {
2   if (err) {
3     console.log(err.message);
4     return;
5   }
6   console.log("Current Data:");
7   console.log(data.toString());
8 });
9
10 console.log("This line prints FIRST");
```

▶ OUTPUT

```
This line prints FIRST
Current Data:
Rohit
Aman
Rahul
Raju
Priya
```

◆ Keeping the order: nest the next task inside the callback

```
1 fs.mkdir("students", (err) => {
2   if (err) return console.log(err.message);
3   console.log("Folder created");
4
5   fs.writeFile("students/data.txt", "Rohit\nAman", (err) => {
6     if (err) return console.log(err.message);
7     console.log("File created");
8
9     fs.readFile("students/data.txt", (err, data) => {
10      if (err) return console.log(err.message);
11      console.log(data.toString());
12    });
13  });
14 });
```

★ KEY POINTS

- Async method names have **no Sync** at the end: `mkdir`, `writeFile`, `readFile`, `appendFile`.
- The result is available **only inside the callback**, not on the next line.
- Always check `err` first.

⚠ COMMON MISTAKES

- Writing async calls one after another (not nested) and expecting them to run in order. `readFile` may finish before `writeFile`, and the output order can change between runs.
- Forgetting `return` after printing the error, so the code continues with an undefined `data`.

🔗 QUICK REVISION: SYNC VS ASYNC

	Sync	Async
Method name	<code>readFileSync</code>	<code>readFile</code>
Blocks next line?	Yes	No
Result	Returned directly	Received inside callback
Error handling	Program throws error	<code>err</code> parameter in callback
Order	Always in written order	Not guaranteed unless nested

☰ UNIT SUMMARY

<code>fs.mkdirSync("folder")</code>	Create a folder
<code>fs.writeFileSync("a.txt", data)</code>	Create / overwrite a file
<code>fs.readFileSync("a.txt", "utf8")</code>	Read a file as text
<code>fs.appendFileSync("a.txt", data)</code>	Add data at the end
<code>fs.readFile("a.txt", (err, data) => {})</code>	Async read (result inside callback)
<code>fs.writeFile("a.txt", data, (err) => {})</code>	Async write

03

UNIT 3

Creating a Server with Node.js

How a server works and how to build one using only the built-in http module.

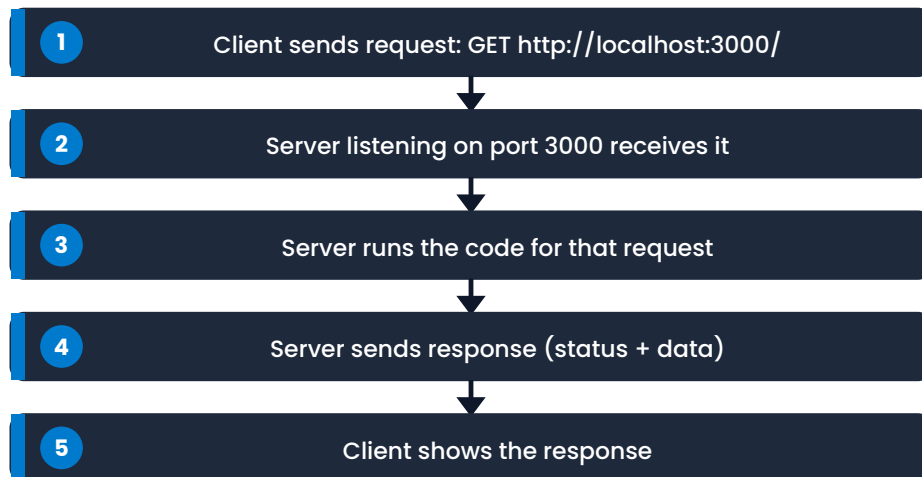
IN THIS UNIT

- 3.1 Client, Server, Request, Response
- 3.2 Creating a Server: http Module
- 3.3 Status Code, Headers and JSON
- 3.4 Routing with req.url and req.method
- 3.5 Why Express

3.1 Client, Server, Request, Response

Term	Meaning
Client	The one that sends a request : browser, Postman, a frontend app
Server	A program that waits for requests and sends responses
Request	Message from client: method (GET/POST...), URL , headers, body
Response	Message from server: status code , headers, data
Port	A number that identifies which program on the computer gets the request (e.g. 3000)
localhost	Your own computer (127.0.0.1)

REQUEST - RESPONSE CYCLE



3.2 Creating a Server: http Module

◆ WHAT IT IS

`http` is a **built-in** module. `http.createServer()` creates a server; the function passed to it runs **on every request**. `server.listen()` starts the server on a port.

{ } SYNTAX

```
1 const http = require("http");
2
3 const server = http.createServer((req, res) => {
4   // runs on EVERY request
5   // req = request, res = response
6 });
7
8 server.listen(PORT, callback);
```

</> EXAMPLE

```
1 const http = require("http");
2
3 const server = http.createServer((req, res) => {
4   res.end("Hello from Node.js server");
5 });
6
7 server.listen(3000, () => {
8   console.log("Server running on http://localhost:3000");
9 });
```

► OUTPUT

```
$ node server.js
Server running on http://localhost:3000

Browser (http://localhost:3000): Hello from Node.js server
```

★ KEY POINTS

- `res.end()` sends the response and **finishes** the request.
- The server keeps running until stopped with **Ctrl + C** in the terminal.
- After changing the code, **restart** the server to see the change.

⚠ COMMON MISTAKES

- Forgetting `res.end()`: the browser keeps loading and never gets a response.
- Starting a second server on the same port: **EADDRINUSE: address already in use**. Stop the old one first.

3.3 Status Code, Headers and JSON

◆ WHAT IT IS

`res.writeHead(statusCode, headers)` sets the status code and headers. The **Content-Type** header tells the client what kind of data is coming.

{ } SYNTAX

```
1 res.writeHead(200, { "Content-Type": "text/plain" }); // plain text
2 res.writeHead(200, { "Content-Type": "application/json" }); // JSON
3 res.end(data);
```

</> EXAMPLE: SENDING JSON

```
1 const http = require("http");
2
3 const server = http.createServer((req, res) => {
4   let user = { name: "Aman", city: "Pune" };
5
6   res.writeHead(200, { "Content-Type": "application/json" });
7   res.end(JSON.stringify(user)); // object -> JSON string
8 });
9
10 server.listen(3000);
```

► OUTPUT

```
{"name": "Aman", "city": "Pune"}
```

⚠ COMMON MISTAKES

- Passing an object directly: `res.end(user)` throws a `TypeError`. `res.end()` accepts only a string (or Buffer), so use `JSON.stringify()`.

3.4 Routing with `req.url` and `req.method`

◆ WHAT IT IS

In the `http` module there is **one** function for all requests. Different pages are handled with `if / else` on `req.url` (the path) and `req.method` (GET, POST, ...).

</> EXAMPLE

```
1  const http = require("http");
2
3  const server = http.createServer((req, res) => {
4    if (req.url === "/" && req.method === "GET") {
5      res.writeHead(200, { "Content-Type": "text/plain" });
6      res.end("Home page");
7    }
8    else if (req.url === "/about" && req.method === "GET") {
9      res.writeHead(200, { "Content-Type": "text/plain" });
10     res.end("About page");
11   }
12   else if (req.url === "/products" && req.method === "GET") {
13     res.writeHead(200, { "Content-Type": "application/json" });
14     res.end(JSON.stringify([{"id": 1, "name": "iPhone 15"}]));
15   }
16   else {
17     res.writeHead(404, { "Content-Type": "text/plain" });
18     res.end("Page not found");
19   }
20 });
21
22 server.listen(3000, () => console.log("Server running..."));
```

► OUTPUT

Request	Status	Response
GET /	200	Home page
GET /about	200	About page
GET /products	200	[{"id":1,"name":"iPhone 15"}]
GET /contact	404	Page not found

★ KEY POINTS

- `req.url` contains the full path including the query string. For `/search?city=Pune`, `req.url` is `/search?city=Pune`, so `req.url === "/search"` is `false`. Reading params, query and body needs extra manual code.

3.5 Why Express

◆ WHAT IT IS

Express is built on top of the `http` module. It does the same work with much less code, and handles params, query and body automatically.

Task	Node.js http	Express
Routing	<code>if / else</code> on <code>req.url</code> and <code>req.method</code>	<code>app.get("/about", ...)</code>
Send JSON	<code>writeHead</code> + <code>JSON.stringify</code> + <code>res.end</code>	<code>res.json({...})</code>
Status code	<code>res.writeHead(404, ...)</code>	<code>res.status(404)</code>
Route params <code>/products/3</code>	Split the URL string manually	<code>req.params.id</code>
Query string <code>?city=Pune</code>	Parse the URL manually	<code>req.query.city</code>
Request body	Read data chunks manually	<code>express.json() + req.body</code>

</> EXAMPLE: THE SAME ROUTE IN BOTH

```

1 // Node.js http
2 if (req.url === "/about" && req.method === "GET") {
3   res.writeHead(200, { "Content-Type": "application/json" });
4   res.end(JSON.stringify({ msg: "About page" }));
5 }
6
7 // Express
8 app.get("/about", (req, res) => {
9   res.json({ msg: "About page" });
10 });

```

≡ UNIT SUMMARY

<code>const http = require("http");</code>	Load the built-in http module
<code>http.createServer((req, res) => {})</code>	Create server; runs on every request
<code>server.listen(3000, () => {})</code>	Start server on port 3000
<code>req.url / req.method</code>	Path / HTTP method of the request
<code>res.writeHead(200, { "Content-Type": "text/plain" })</code>	Set status code and headers
<code>res.end(JSON.stringify(obj))</code>	Send response and finish

04

UNIT 4

Express Basics

Building servers with Express: routes, responses, params, query and body.

IN THIS UNIT

4.1 What is Express

4.2 Routes

4.3 Sending Responses

4.4 Route Params: req.params

4.5 Query String: req.query

4.6 Request Body: req.body

4.1 What is Express

◆ WHAT IT IS

Express is a Node.js **framework** for building servers and APIs. It is built on top of the **http** module and removes the manual work from Unit 3: no **if / else** on **req.url**, no **JSON.stringify**, no manual headers.

{ } SYNTAX

```
1 npm install express           // run once in terminal
2
3 let express = require("express");
4 let app = express();           // create the app
5
6 app.listen(PORT, callback);    // start the server
```

</> EXAMPLE

```
1 let express = require("express");
2 let app = express();
3
4 app.get("/", (req, res) => {
5   res.send("Hello from server");
6 });
7
8 app.listen(3000, () => {
9   console.log("server.....");
10 });
```

▶ RESULT

Terminal shows **server.....**. Opening **http://localhost:3000/** in the browser shows **Hello from server**.

4.2 Routes

◆ WHAT IT IS

A **route** connects an HTTP method and a path to a function. When a request matches both, that function runs.

{ } SYNTAX

```
1 app.METHOD(path, (req, res) => {  
2   // req = request (data coming IN)  
3   // res = response (data going OUT)  
4 });
```

Method	Used for	Example
GET	Reading data	<code>app.get("/products", ...)</code>
POST	Creating new data	<code>app.post("/products", ...)</code>
PUT	Updating existing data	<code>app.put("/products/:id", ...)</code>
DELETE	Removing data	<code>app.delete("/products/:id", ...)</code>

A browser address bar can only send GET requests. Use **Postman** (or Thunder Client) to test POST, PUT and DELETE.

4.3 Sending Responses

{ } SYNTAX

Method	What it does
<code>res.send("text")</code>	Sends text (or any data)
<code>res.json({ msg: "done" })</code>	Sends data in JSON format
<code>res.status(404)</code>	Sets the status code. Chain it: <code>res.status(404).json({...})</code>

</> EXAMPLE

```
1 app.get("/test", (req, res) => {  
2   res.status(200).json({ msg: "hello" });  
3 });
```

► OUTPUT (IN BROWSER / POSTMAN)

```
{ "msg": "hello" }
```

★ KEY POINTS

- A request can get **only one** response.
- Use `return res...` when you want to stop the function after sending a response (for example, inside an `if`).

⚠ COMMON MISTAKES

- Sending two responses in one request gives the error **Cannot set headers after they are sent to the client**. This happens when `return` is missing inside an `if` block.

4.4 Route Params: req.params

◆ WHAT IT IS

A **route param** is a variable part of the path, written with `:` in the route. It is used to identify **one specific item**, like one product by its id.

{ } SYNTAX

```
1 app.get("/products/:id", (req, res) => {
2   let { id } = req.params;    // same as: let id = req.params.id
3 });
```

</> EXAMPLE

```
1 app.get("/new/:id", (req, res) => {
2   let { id } = req.params;
3   console.log(req.params);
4   res.send(id);
5 });
```

▶ OUTPUT

Request: `http://localhost:3000/new/7`

```
Terminal: { id: '7' }
Browser:  7
```

★ KEY POINTS

- The value is always a **string** (`'7'`, not `7`). Convert with `Number(id)` before comparing with `===`.
- The name after `:` becomes the key in `req.params`.

4.5 Query String: req.query

◆ WHAT IT IS

A **query string** comes after **?** in the URL as **key=value** pairs joined by **&**. It is used for **search and filters**.

{ } SYNTAX

```
1 // URL: /search?firstName=om&lastName=jain
2 app.get("/search", (req, res) => {
3   let { firstName, lastName } = req.query;
4 });
```

</> EXAMPLE

```
1 app.get("/search", (req, res) => {
2   console.log(req.query);
3   res.send("ok");
4 });
```

► OUTPUT

Request: `http://localhost:3000/search?firstName=om&lastName=jain`

```
{ firstName: 'om', lastName: 'jain' }
```

★ KEY POINTS

- The route path does **not** mention the query keys. Only `/search` is written.
- Query values are also **strings**.

4.6 Request Body: req.body

◆ WHAT IT IS

The **body** carries data sent with POST and PUT requests, usually in JSON format. Express does not read it by default; `express.json()` must be added.

{ } SYNTAX

```
1 app.use(express.json()); // write once, before the routes
2
3 app.post("/about", (req, res) => {
4   console.log(req.body);
5 });
```

</> EXAMPLE

```

1 app.use(express.json());
2
3 app.post("/about", (req, res) => {
4   console.log(req.body);
5   res.status(200).json({ msg: "hello" });
6 });

```

▶ OUTPUT

Postman: POST `http://localhost:3000/about`, Body -> raw -> JSON:

```

Sent body: { "name": "Aman", "age": 22 }
Terminal:  { name: 'Aman', age: 22 }

```

⚠ COMMON MISTAKES

- Forgetting `app.use(express.json())` makes `req.body` undefined.
- Selecting **Text** instead of **JSON** in Postman sends the body in the wrong format.

🔄 Quick Revision: params vs query vs body

	req.params	req.query	req.body
Where in request	Inside the path	After <code>?</code> in URL	Request body
Example	<code>/products/3</code>	<code>/search?category=mobile</code>	<code>{ "name": "Aman" }</code>
Route written as	<code>/products/:id</code>	<code>/search</code>	<code>/products</code>
Used for	One specific item	Search / filter	Sending new data
Extra setup	None	None	<code>express.json()</code>

≡ UNIT SUMMARY

<code>let app = express();</code>	Create the app
<code>app.use(express.json());</code>	Enable <code>req.body</code> for JSON
<code>app.get("/path", (req, res) => {})</code>	Route: method + path + handler
<code>app.listen(3000, () => {})</code>	Start the server
<code>req.params.id</code>	Value from <code>/products/:id</code>
<code>req.query.city</code>	Value from <code>?city=Pune</code>
<code>req.body</code>	Data sent in POST / PUT
<code>res.status(404).json({ msg: "... " })</code>	Send status + JSON

05

UNIT 5

REST API and CRUD

A complete CRUD API on a products array, status codes and CORS.

IN THIS UNIT

- 5.1 What is a REST API
- 5.2 The Data (In-Memory Array)
- 5.3 GET All Products
- 5.4 GET One Product by id: find()
- 5.5 GET by Category: filter()
- 5.6 POST: Add a Product with push()
- 5.7 PUT: Update a Product
- 5.8 DELETE: findIndex() + splice()
- 5.9 CORS

5.1 What is a REST API

◆ WHAT IT IS

A **REST API** is a set of routes where the **path** names the **resource** (like `/products`) and the **HTTP method** names the **action**. **CRUD** = Create, Read, Update, Delete.

Method	Route	Action	CRUD
GET	<code>/products</code>	Get all products	Read
GET	<code>/products/3</code>	Get product with id 3	Read
GET	<code>/products?category=mobile</code>	Get products of one category	Read
POST	<code>/products</code>	Add a new product	Create
PUT	<code>/products/3</code>	Update product with id 3	Update
DELETE	<code>/products/3</code>	Delete product with id 3	Delete

5.2 The Data (In-Memory Array)

```
JavaScript
1 let express = require("express");
2 let cors = require("cors");
3 let app = express();
4
5 app.use(express.json());
6 app.use(cors());
7
8 let products = [
9   { id: 1, name: "iPhone 15", category: "mobile", price: 69999, stock: 10 },
10  { id: 2, name: "Galaxy S24", category: "mobile", price: 64999, stock: 8 },
11  { id: 3, name: "MacBook Air", category: "laptop", price: 99999, stock: 5 },
12  { id: 4, name: "Dell XPS 14", category: "laptop", price: 89999, stock: 7 },
13  { id: 5, name: "AirPods Pro", category: "headphones", price: 24999, stock: 15 },
14  { id: 6, name: "Sony XMS", category: "headphones", price: 29999, stock: 12 }
15 ];
```

The data lives in a variable, so it resets every time the server restarts.

5.3 GET All Products

```
JavaScript
1 app.get("/products", (req, res) => {
2   res.status(200).json({ data: products });
3 });
```

5.4 GET One Product by id: find()

```
JavaScript
1 app.get("/products/:id", (req, res) => {
2   let { id } = req.params;
3
4   // find() returns the FIRST matching object, or undefined
5   let singleData = products.find((a) => a.id === Number(id));
6
7   if (!singleData) {
8     return res.status(404).json({ msg: "Product not found" });
9   }
10  res.status(200).json({ data: singleData });
11 });
```

► OUTPUT

```
Terminal / Output
GET /products/3 -> { "data": { "id": 3, "name": "MacBook Air", ... } }
GET /products/99 -> { "msg": "Product not found" } (status 404)
```

5.5 GET by Category: filter()

```
JavaScript
1 app.get("/search", (req, res) => {
2   let { category } = req.query;
3
4   // filter() returns an ARRAY of all matches, or [] if none
5   let data = products.filter((a) => a.category === category);
6
7   if (data.length === 0) {
8     return res.status(404).json({ msg: "No products found" });
9   }
10  res.status(200).json({ data: data });
11 });
```

► OUTPUT

```
Terminal / Output
GET /search?category=laptop -> { "data": [ {id: 3, ...}, {id: 4, ...} ] }
GET /search?category=tv -> { "msg": "No products found" } (status 404)
```

⚠ COMMON MISTAKES

- Using `if (!data)` with `filter()`. An empty array `[]` is **truthy**, so `![]` is **false** and the 404 never runs. Check `data.length === 0`.
- Using `find()` here returns only the first laptop, not all laptops.

5.6 POST: Add a Product with `push()`

```
1 app.post("/products", (req, res) => {
2   let obj = { ...req.body }; // copy all fields from body
3   products.push(obj);
4   res.status(201).json({ msg: "Product added", data: obj });
5 });
```

▶ OUTPUT

```
Body:    { "id": 7, "name": "iPad", "category": "tablet", "price": 34999, "stock": 6 }
Response: { "msg": "Product added", "data": { "id": 7, ... } } (status 201)
```

`...req.body` is the **spread operator**: it copies every key of `req.body` into the new object.

5.7 PUT: Update a Product

```
1 app.put("/products/:id", (req, res) => {
2   let { id } = req.params;
3   let { stock } = req.body;
4
5   let data = products.find((a) => a.id === Number(id));
6
7   if (!data) {
8     return res.status(404).json({ msg: "Product not found" });
9   }
10  data.stock = stock; // changes the object inside the array
11  res.status(200).json({ msg: "Product updated", data: data });
12 });
```

▶ OUTPUT

```
PUT /products/1 Body: { "stock": 100 }
-> { "msg": "Product updated", "data": { "id": 1, ..., "stock": 100 } }
```

⚠ COMMON MISTAKES

- Skipping the not-found check. If the id does not exist, `data` is **undefined** and `data.stock = ...` crashes with **Cannot set properties of undefined**.

5.8 DELETE: findIndex() + splice()

```

1  app.delete("/products/:id", (req, res) => {
2    let { id } = req.params;
3
4    // findIndex() returns the position, or -1 if not found
5    let index = products.findIndex((a) => a.id === Number(id));
6
7    if (index === -1) {
8      return res.status(404).json({ msg: "Product not found" });
9    }
10   let deleted = products.splice(index, 1); // remove 1 item at index
11   res.status(200).json({ msg: "Product deleted", data: deleted });
12 });

```

► OUTPUT

```

DELETE /products/6
-> { "msg": "Product deleted", "data": [ { "id": 6, "name": "Sony XM5", ... } ] }

```

5.9 CORS

◆ WHAT IT IS

CORS (Cross-Origin Resource Sharing): the browser blocks a request when the frontend and backend run on a different **origin** (different port or domain), for example frontend on **localhost:5173** and backend on **localhost:3000**. The **cors** package tells the browser that such requests are allowed.

{ } SYNTAX

```

1  npm install cors
2
3  let cors = require("cors");
4  app.use(cors());

```

CORS errors appear only in the **browser**. Postman is not a browser, so it works without CORS.

🔄 Quick Revision: Array Methods in APIs

Method	Returns	If nothing matches	Used in
find()	First matching item	undefined	GET by id, PUT
filter()	Array of all matches	[] (empty array)	Search by category
findIndex()	Position of first match	-1	DELETE
push()	Adds item at end	-	POST
splice(i, 1)	Removes 1 item at position i	-	DELETE

🔄 QUICK REVISION: STATUS CODES

Code	Meaning	When to use
200	OK	Request succeeded
201	Created	New data added (POST)
401	Unauthorized	No token / invalid token
403	Forbidden	Logged in, but not allowed (wrong role)
404	Not Found	Item does not exist
500	Server Error	Something broke on the server

☰ UNIT SUMMARY

<code>products.find((a) => a.id === Number(id))</code>	GET one / PUT: first match
<code>products.filter((a) => a.category === category)</code>	GET search: all matches
<code>products.push({ ...req.body })</code>	POST: add item
<code>products.findIndex((a) => a.id === Number(id))</code>	DELETE: get position
<code>products.splice(index, 1)</code>	DELETE: remove 1 item
<code>if (data.length === 0) {}</code>	Empty check for <code>filter()</code>
<code>app.use(cors());</code>	Allow frontend on another port

06

UNIT 6

MongoDB with Mongoose

Storing users in MongoDB using Mongoose models.

IN THIS UNIT

- 6.1 Connecting to MongoDB
- 6.2 Schema and Model
- 6.3 async / await with Database Calls
- 6.4 Creating a Document: new + save()
- 6.5 Finding a Document: findOne()
- 6.6 Updating a Document
- 6.7 Query Operator: \$gt

6.1 Connecting to MongoDB

◆ WHAT IT IS

MongoDB is a database that stores data as **documents** (JSON-like objects) inside **collections**. **Mongoose** is a Node.js package used to connect to MongoDB and work with its data.

{ } SYNTAX

```
1 npm install mongoose
2
3 let mongoose = require("mongoose");
4 mongoose.connect("mongodb://127.0.0.1:27017/dbName")
5   .then(() => console.log("db....."));
```

★ KEY POINTS

- **27017** is MongoDB's default port. **db** at the end of the URL is the database name.
- **mongoose.connect()** returns a promise, so **.then()** runs after the connection succeeds.

6.2 Schema and Model

◆ WHAT IT IS

A **schema** defines the fields and their types for a document. A **model** is created from the schema and is used to add, find and update documents.

</> EXAMPLE: DB/DB.JS

```
1 let mongoose = require("mongoose");
2
3 let userSchema = new mongoose.Schema({
4   name: String,
5   email: String,
6   passWord: String,
7   role: { type: String, default: "user" },
8   resetToken: String,
9   resetTokenExpiry: Date
10 });
11
12 let User = mongoose.model("User", userSchema); // collection: users
13 module.exports = User;
```

</> USING THE MODEL IN THE MAIN FILE

```
1 let User = require("./db/db.js");
```

Mongoose converts the model name **User** to the collection name **users** (lowercase + plural).

6.3 async / await with Database Calls

◆ WHAT IT IS

Database calls take time and return a **promise**. **await** pauses the function until the result arrives. **await** works only inside a function marked **async**.

{ } SYNTAX

```
1 app.post("/route", async (req, res) => {
2   let result = await Model.someMethod();
3 });
```

⚠ COMMON MISTAKES

- Forgetting `await`: the variable holds a pending **Promise**, not the data.
- Using `await` inside a function without `async` gives a syntax error.

6.4 Creating a Document: new + save()

```
1 let userInfo = new User({
2   name: "Aman",
3   email: "aman@gmail.com",
4   passWord: "hashedPasswordHere"
5 });
6 await userInfo.save();           // stores it in the users collection
```

MongoDB automatically adds a unique `_id` to every document.

6.5 Finding a Document: findOne()

```
1 let user = await User.findOne({ email: "aman@gmail.com" });
```

▶ OUTPUT

```
Found:      { _id: ObjectId('...'), name: 'Aman', email: 'aman@gmail.com', ... }
Not found:  null
```

★ KEY POINTS

- `findOne()` returns **one document** or `null`. Always check for `null` before using the result.

6.6 Updating a Document

```
1 let user = await User.findOne({ email: "aman@gmail.com" });
2 user.name = "Aman Jain";           // change the field
3 await user.save();                 // save the change to the database
```

6.7 Query Operator: \$gt

◆ WHAT IT IS

`$gt` means **greater than**. It is written inside the condition object of a query.

{ } SYNTAX

```
1 { fieldName: { $gt: value } }
```

</> EXAMPLE

```
1 // Find the user whose token expiry time is still in the future
2 let user = await User.findOne({
3   resetToken: token,
4   resetTokenExpiry: { $gt: Date.now() }
5 });
```

`Date.now()` gives the current time in **milliseconds**. If the expiry is greater than now, the token is still valid.

≡ UNIT SUMMARY

<code>mongoose.connect("mongodb://127.0.0.1:27017/db")</code>	Connect to database
<code>new mongoose.Schema({ name: String })</code>	Define fields
<code>mongoose.model("User", userSchema)</code>	Create model
<code>await new User({ ... }).save()</code>	Create document
<code>await User.findOne({ email })</code>	Find one (or <code>null</code>)
<code>user.name = 'X'; await user.save();</code>	Update document
<code>{ field: { \$gt: value } }</code>	Greater than condition

07

UNIT 7

Authentication and Authorization

Signup, login, password hashing, JWT tokens and protected routes.

IN THIS UNIT

[7.1 Authentication vs Authorization](#)[7.2 Password Hashing: bcryptjs](#)[7.3 Signup](#)[7.4 JWT \(JSON Web Token\)](#)[7.5 Login](#)[7.6 Middleware](#)[7.7 Auth Middleware](#)[7.8 Role Check Middleware](#)

7.1 Authentication vs Authorization

	Authentication	Authorization
Question	Who are you?	What are you allowed to do?
Done by	Signup / Login + JWT token	Role check middleware
Fails with	401 Unauthorized	403 Forbidden

7.2 Password Hashing: bcryptjs

◆ WHAT IT IS

Passwords are never stored as plain text. **Hashing** converts a password into a fixed-length scrambled string. Hashing is **one-way**: the original password cannot be recovered from the hash, it can only be compared.

{ } SYNTAX

```
1 npm install bcryptjs
2 let bcryptjs = require("bcryptjs");
3
4 let hashed = await bcryptjs.hash(plainPassword, 10); // 10 = salt rounds
5 let isValid = await bcryptjs.compare(plainPassword, hashed); // true / false
```

</> EXAMPLE

```
1 let hashed = await bcryptjs.hash("abc123", 10);
2 console.log(hashed);
3
4 console.log(await bcryptjs.compare("abc123", hashed));
5 console.log(await bcryptjs.compare("wrong", hashed));
```

► OUTPUT

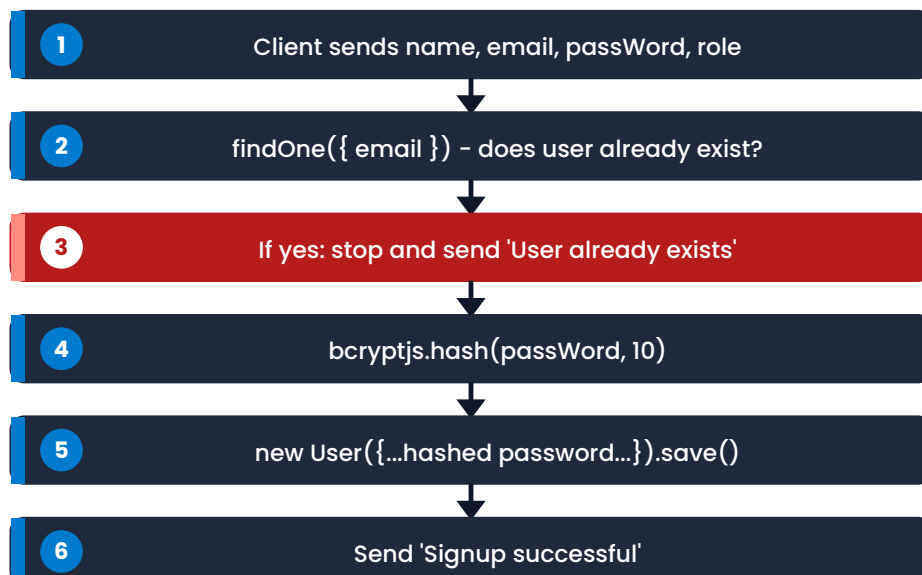
```
$2b$10$Xk3... (a 60-character string, different on every run)
true
false
```

★ KEY POINTS

- The same password gives a **different hash** every time (because of the random salt), but `compare()` still works.
- The order in `compare()` is fixed: **plain password first, hash second**.

7.3 Signup

SIGNUP FLOW



```
JavaScript
1 app.post("/signup", async (req, res) => {
2   let { name, email, passWord, role } = req.body;
3
4   let findData = await User.findOne({ email });
5   if (findData) {
6     return res.status(400).send("User already exists");
7   }
8
9   let hashedP = await bcryptjs.hash(passWord, 10);
10
11  let userInfo = new User({
12    name, // same as name: name
13    email,
14    passWord: hashedP,
15    role: role || "user" // default role is "user"
16  });
17  await userInfo.save();
18  res.status(201).send("Signup successful");
19 });
```

`role || "user"`: if `role` is not sent in the body, it becomes `"user"`.

7.4 JWT (JSON Web Token)

◆ WHAT IT IS

A **JWT** is a signed token that the server gives to the user after login. The user sends it with every protected request. The server checks the signature to confirm the token is genuine and has not been changed.

{ } SYNTAX

```
1 npm install jsonwebtoken
2 let jwt = require("jsonwebtoken");
3
4 let token = jwt.sign(payload, secretKey); // create token
5 let decoded = jwt.verify(token, secretKey); // check token, returns payload
```

≡ STRUCTURE

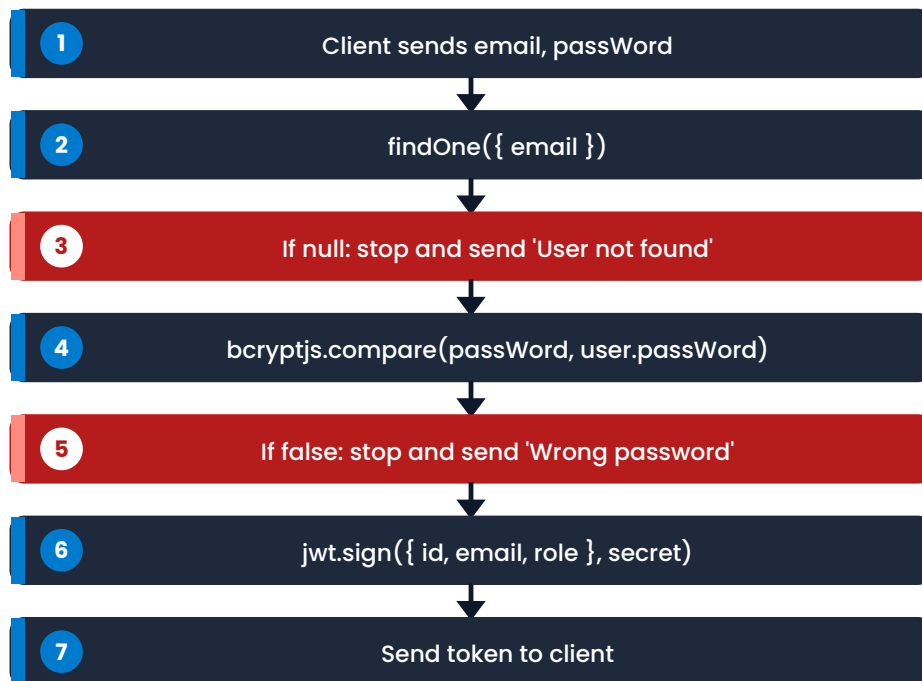
```
1 eyJhbGciOiJIUzI1NiJ9 . eyJpZCI6IjY2Zj... . SflKxwRJSMeKKF2QT4...
2      HEADER                PAYLOAD                SIGNATURE
```

★ KEY POINTS

- The **payload** holds user info like `id`, `email`, `role`. Never put the password in it; the payload can be read by anyone.
- The **secret key** must be the same in `sign()` and `verify()`. It must be kept private (in a `.env` file in real projects).
- `jwt.verify()` throws an error if the token is wrong or changed.

7.5 Login

LOGIN FLOW



```
JavaScript
1 app.post("/login", async (req, res) => {
2   let { email, passWord } = req.body;
3
4   let findData = await User.findOne({ email });
5   if (!findData) {
6     return res.status(404).send("User not found");
7   }
8
9   let validP = await bcryptjs.compare(passWord, findData.passWord);
10  if (!validP) {
11    return res.status(401).send("Wrong password");
12  }
13
14  let token = jwt.sign(
15    { id: findData._id, email: findData.email, role: findData.role },
16    "mySecretKey"
17  );
18  res.json({ msg: "Login successful", token: token });
19 });
```

⚠ COMMON MISTAKES

- Skipping the `null` check: if the email does not exist, `findData.passWord` crashes with **Cannot read properties of null**.

7.6 Middleware

◆ WHAT IT IS

A **middleware** is a function that runs **between** the request and the route handler. It receives `(req, res, next)`. Calling `next()` passes the request to the next function. Not calling `next()` (and sending a response instead) stops the request there.

{ } SYNTAX

```
1 let myMiddleware = (req, res, next) => {  
2   // checks here  
3   next();           // go to the next function  
4 };  
5  
6 app.get("/path", myMiddleware, (req, res) => { ... });
```

`app.use(express.json())` and `app.use(cors())` are also middleware, applied to **every** route.

7.7 Auth Middleware

```
1 let auth = (req, res, next) => {  
2   let token = req.headers.authorization;  
3  
4   if (!token) {  
5     return res.status(401).send("No token, please login");  
6   }  
7   try {  
8     let decoded = jwt.verify(token, "mySecretKey");  
9     req.user = decoded; // { id, email, role } available to next functions  
10    next();  
11  } catch (err) {  
12    return res.status(401).send("Invalid token");  
13  }  
14 };
```

In Postman: Headers tab -> key `Authorization`, value = the token from login.

7.8 Role Check Middleware

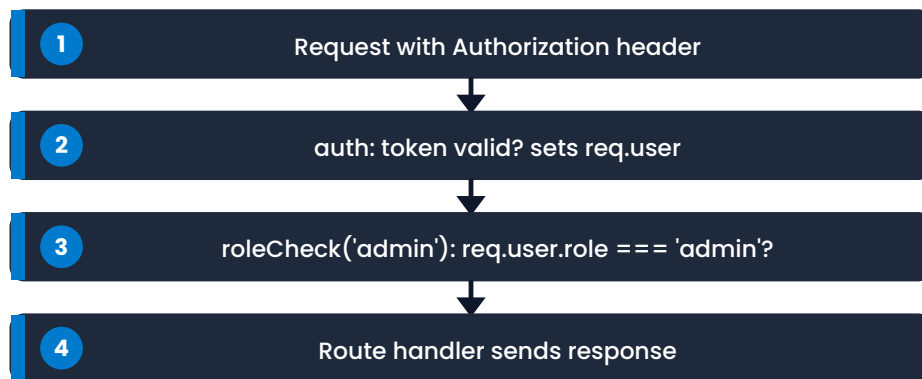
◆ WHAT IT IS

`roleCheck` is a function that **returns a middleware**. This allows passing a role: `roleCheck("admin")`, `roleCheck("teacher")`, and so on.

```

1  let roleCheck = (role) => {
2    return (req, res, next) => {
3      if (req.user.role !== role) {
4        return res.status(403).send("Access denied");
5      }
6      next();
7    };
8  };
9
10 app.get("/admin", auth, roleCheck("admin"), (req, res) => {
11   res.send("Hello, only admin can access me!");
12 });
  
```

REQUEST TO GET /ADMIN



► OUTPUT

Situation	Response
No token	401 - No token, please login
Wrong / changed token	401 - Invalid token
Valid token, role = user	403 - Access denied
Valid token, role = admin	200 - Hello, only admin can access me!

⚠ COMMON MISTAKES

- Placing `roleCheck` before `auth`: `req.user` does not exist yet, so the code crashes.
- Calling `jwt.verify()` without try/catch: an invalid token crashes the request.

UNIT SUMMARY

<code>await bcryptjs.hash(passWord, 10)</code>	Hash a password
<code>await bcryptjs.compare(plain, hashed)</code>	Check password: true / false
<code>jwt.sign({ id, email, role }, secret)</code>	Create token at login
<code>jwt.verify(token, secret)</code>	Check token, returns payload
<code>req.headers.authorization</code>	Token sent by client
<code>(req, res, next) => { next(); }</code>	Middleware shape
<code>app.get("/admin", auth, roleCheck("admin"), handle r)</code>	Chain middleware

UNIT 8

08

Forgot and Reset Password

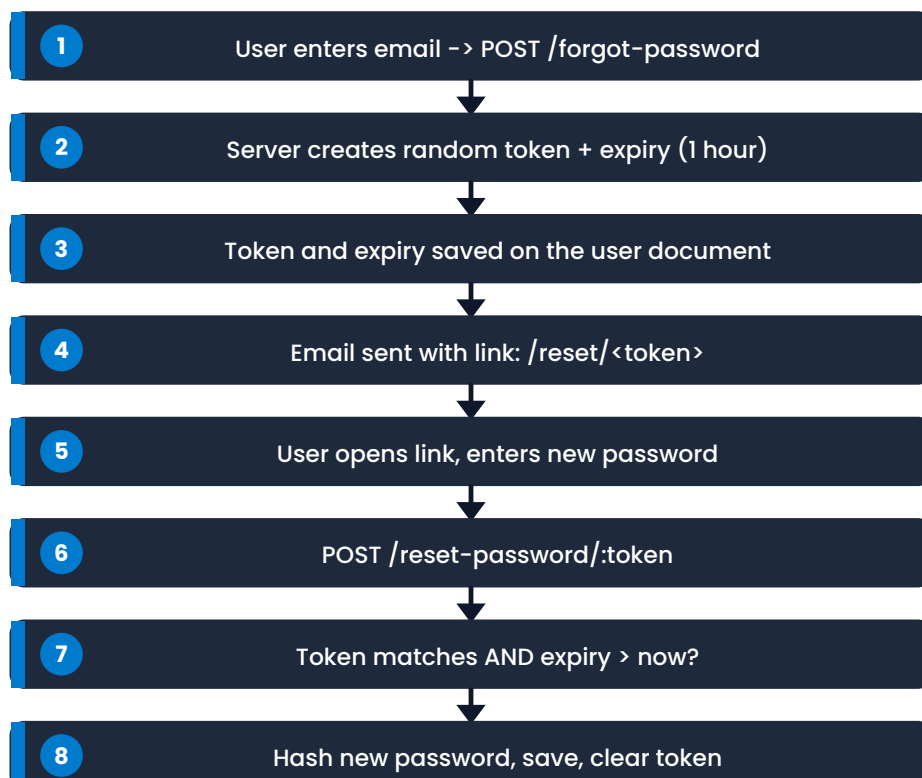
Complete forgot-password flow with a random token, expiry and email link.

IN THIS UNIT

- 8.1 The Complete Flow
- 8.2 Generating a Random Token: crypto
- 8.3 Sending Email: sendEmail Utility
- 8.4 Route: /forgot-password
- 8.5 Route: /reset-password/:token

8.1 The Complete Flow

FORGOT / RESET PASSWORD FLOW



8.2 Generating a Random Token: crypto

◆ WHAT IT IS

`crypto` is a **built-in** Node.js module. `randomBytes(20)` creates 20 random bytes; `.toString("hex")` converts them into a 40-character string.



JavaScript

```
1 let crypto = require("crypto");
2 let resetToken = crypto.randomBytes(20).toString("hex");
3 console.log(resetToken);
```

▶ OUTPUT



Terminal / Output

```
9f2c4e7a1b8d03f6c5e2a9b7d4f1e0c3a8b6d2f5 (different every run)
```

8.3 Sending Email: sendEmail Utility

The email logic is kept in a separate file `utils/sendMail.js` using the `nodemailer` package, and exported as `sendEmail(to, subject, text)`.



utils/sendMail.js

```
1 // utils/sendMail.js
2 let nodemailer = require("nodemailer");
3
4 let transporter = nodemailer.createTransport({
5   service: "gmail",
6   auth: { user: "yourmail@gmail.com", pass: "your-app-password" }
7 });
8
9 let sendEmail = async (to, subject, text) => {
10   await transporter.sendMail({ from: "yourmail@gmail.com", to, subject, text });
11 };
12
13 module.exports = { sendEmail };
```



JavaScript

```
1 // main file
2 let { sendEmail } = require("../utils/sendMail.js");
```

8.4 Route: /forgot-password

```
JavaScript
1 app.post("/forgot-password", async (req, res) => {
2   let { email } = req.body;
3   try {
4     let user = await User.findOne({ email });
5     if (!user) {
6       return res.status(404).send("User not found");
7     }
8
9     let resetToken = crypto.randomBytes(20).toString("hex");
10    user.resetToken = resetToken;
11    user.resetTokenExpiry = Date.now() + 3600000; // 3600000 ms = 1 hour
12    await user.save();
13
14    let resetUrl = `http://localhost:5173/reset/${resetToken}`;
15    await sendEmail(user.email, "Password Reset Request",
16      `Click the link below to reset your password:\n\n${resetUrl}`);
17
18    res.status(200).send("Password reset email sent");
19  } catch (error) {
20    res.status(500).send("Error: " + error.message);
21  }
22 });
```

8.5 Route: /reset-password/:token

```
JavaScript
1 app.post("/reset-password/:token", async (req, res) => {
2   let { newP } = req.body;
3   let { token } = req.params;
4
5   let user = await User.findOne({
6     resetToken: token,
7     resetTokenExpiry: { $gt: Date.now() } // not expired
8   });
9
10  if (!user) {
11    return res.status(400).send("Invalid or expired link");
12  }
13
14  user.passWord = await bcryptjs.hash(newP, 10);
15  user.resetToken = undefined; // link cannot be used again
16  user.resetTokenExpiry = undefined;
17  await user.save();
18  res.send("Password reset successful");
19 });
```

★ KEY POINTS

- The token travels in the **URL** (`req.params`), the new password travels in the **body**.
- The new password is **hashed** before saving, exactly like signup.
- Clearing the token makes the reset link **single-use**.

UNIT SUMMARY

<code>crypto.randomBytes(20).toString("hex")</code>	Random reset token
<code>Date.now() + 3600000</code>	Expiry: 1 hour from now
<code>await sendEmail(to, subject, text)</code>	Send the reset link
<code>{ resetTokenExpiry: { \$gt: Date.now() } }</code>	Token not expired
<code>user.resetToken = undefined;</code>	Make link single-use

09

UNIT 9

Error Handling with try / catch

Handling runtime errors safely with try / catch.

IN THIS UNIT

9.1 try / catch

9.1 try / catch

◆ WHAT IT IS

Code that may fail is placed inside `try`. If any line throws an error, the remaining lines of `try` are **skipped** and control jumps to `catch`, which receives the error object.

{ } SYNTAX

```
1 try {
2   // code that may fail
3 } catch (err) {
4   // runs only if an error was thrown
5   // err.message = the error text
6 }
```

</> EXAMPLE

```
1 app.get("/error", (req, res) => {
2   try {
3     let user = null;
4     console.log(user.name); // throws error here
5     console.log("hehehe"); // skipped
6     res.send("hello"); // skipped
7   } catch (err) {
8     res.status(500).send("Something went wrong: " + err.message);
9   }
10 });
```

▶ OUTPUT



Terminal / Output

```
Something went wrong: Cannot read properties of null (reading 'name')
```

★ KEY POINTS

- Without try/catch, an error inside a route can crash the request.
- In `async` routes, wrap the `await` database calls in try/catch, as in `/forgot-password`.
- Send a proper status code (usually 500) from `catch`.

⚠ COMMON MISTAKES

- `res.send("error", err)`: `res.send()` takes **one** argument only. Combine them:
`res.send("error: " + err.message).`

≡ UNIT SUMMARY

<code>try { ... } catch (err) { ... }</code>	Catch errors instead of crashing
<code>err.message</code>	Error text
<code>res.status(500).send(err.message)</code>	Send error response